AI-Assisted Discovery of Physics Laws: Practical Improvements and Considerations

Prasham Shah, Harinarayan Asoori Sriram, Ishani Bakshi, Logan Miller, Aarav Khatri,
Katherine Estevanell, Henry Gaus, Oliver Kahng, Chloe Krawczak, Joshua Moore, Kaitlin Zhang

The Governor's School of New Jersey Program in the Sciences, Drew University
Madison, New Jersey, United States of America

Emails: prasham31@gmail.com, harinarayansriram@gmail.com, ishanibakshi73@gmail.com, logandmill12@gmail.com, aaravkhatri19@gmail.com, kestevanell@gmail.com, gaushenry@gmail.com, olivergkahng@gmail.com, chloekrawczakcollege@gmail.com, moore.joshua.c@gmail.com, pinkballerina18@gmail.com

***Abstract*—The discovery of fundamental physics laws has traditionally required extensive human analysis of experimental data. However, as the scope of the field grows, so does the complexity and volume of the data, and, in turn, the difficulty of finding underlying physical laws and their mathematical formulations. In this study, we thus evaluated the utility of the foundation laid by the "AI Feynman" equation-fitting system [1] of sequential linear regression, symmetry searching, and data table transformations. We also tested those principles on more realistic data, with added noise and varying numerical ranges, neglected in prior works. To do so, we first replicated from scratch the core system of "AI Feynman" and tested our system on synthetic datasets representing various polynomial and transcendental physics equations. We recorded the system's ability to identify translational, sum, and scale symmetries, as well as its performance under noise. Our results show that the algorithm generally performs well on low-noise datasets, though the complexity of the underlying equation affects the impact of noise. We also found that the numerical range of data should be considered in the symmetry detection subroutine. To address the latter point, we developed an adaptive formula that significantly improves symmetry detection across numerical scales, which could be further explored in future research. Overall, this work replicates the core physics-based equation-finding approach of "AI Feynman," highlights its current limitations, tests novel and powerful improvements, and provides direction for future efforts.**

Keywords—equation fitting, noise, symmetries, regression, AI Feynman

I. Introduction

Since the 17th century, physics has sought to understand the world in terms of concise mathematical equations. For instance, consider Newton's Law of Universal Gravitational Attraction, often expressed as

$$F_g = \frac{Gm_0m_1}{(x_0-x_1)^2} \quad (1)$$

For our purposes, the definitions of each variable in this equation, as well as in any other equations presented in this paper, is not particularly important. What is important is that manually deriving this equation from data presents several obstacles. First is the relation's inherent complexity, as it includes four variables and a constant, which could potentially combine in countless ways. Then, there is the issue of noise, or fluctuations that displace data points from their theoretically accurate values. As such, the following question arises: Can we robustly and adaptively automate the discovery of the equations underlying realistic datasets of varying scales and noise levels?

II. Literature Review

This field of automated equation discovery can be traced back to early efforts by Pat Langley's BACON systems [2]. These systems pioneered heuristic-based discovery by observing correlations and constructing new terms. More modern approaches, like genetic programming, explore vast expression spaces by evolving mathematical expressions as tree structures, though unconstrained versions can yield overly complex models [3]. More recently, neural network and deep learning-guided methods, including Deep Symbolic Regression and neural-guided Monte Carlo Tree Search systems, along with emerging LLMs, offer powerful and sophisticated pattern-recognition capabilities [4]. However, a key concern arises: the newer of the aforementioned systems tend to stack layers of complexity such that expanding upon them or adapting them to new situations becomes unwieldy. Moreover, many modern ML models operate as "black boxes," providing accurate predictions without revealing the human-interpretable underlying mechanisms. In this landscape, our work is heavily inspired by Udrescu and Tegmark's "AI Feynman" [1], which prioritizes both simplicity and interpretability while delivering exceptional performance. The physics-informed approach of "AI Feynman" leverages principles like symmetry searching and data table modifications to simplify the data and derive their underlying function. "AI Feynman" successfully identifies 100 classic physics equations from *The Feynman Lectures on Physics* [5].

However, the "AI Feynman" paper fails to address how symmetry detection could be adapted to different data scales (i.e., data of different numerical ranges). Moreover, the paper neglects to detail the specific effects of different noise levels (i.e., exactly how noise changes the ultimate equation found). In our work, we thus expanded upon their ideas by examining the effects of noise in more detail and by crafting a novel method to adapt to datasets of varying numerical range. We also tested our system on new equations to confirm the algorithm's utility beyond the testing set the "AI Feynman" paper used.

We thank the Office of the Secretary of Higher Education of New Jersey, the Overdeck Family Foundation, Novartis, Bristol Myers Squibb, and the Governor's School of New Jersey Program in the Sciences (GSNJS) alumnae and parents of alumnae for sponsoring the 42nd GSNJS and making our research possible.

III. Method

```
function SOLVE(M, V)
    s ← LinearRegression(M, V)
    if s.score ≥ T then return s
    end if
    seq ← [PolyNoCross, AddFunctional, PolyWithCross, AddFunctional]
    for each f in seq do
        repeat
            (M, V, k) ← FINDSYMMETRIES(M, V)
        until k = 0
        (M, V) ← f(M, V)
        s ← LinearRegression(M, V)
        if s.score ≥ T then return s
        end if
    end for
    repeat
        (M, V, k) ← FINDSYMMETRIES(M, V)
    until k = 0
    return s
end function
```

Fig. 1. Pseudocode for the main equation-fitting algorithm, which tries basic linear regression, polynomial regression without cross terms, functional features, polynomial regression with cross terms, and functional features once again, interweaving symmetry checks between each step. ***M*** is a matrix of all the observations (columns are variables). *V* is an array with the names of all the independent variables. Each of the transformation functions returns an updated matrix, variables list, and, for symmetry searching, the symmetry-found count.

Our system reimplements from scratch a few key portions of the "AI Feynman" algorithm by Udrescu and Tegmark [1]. Our algorithm's sole purpose is to find the relation between many independent variables and a single dependent variable. The core strategy, which we term the "data table approach," is to systematically expand the matrix of independent variables with new columns representing transformed features. The goal is to find a feature set where the dependent variable can be modeled as a linear combination of those features. This process is applied iteratively, progressing through stages of increasing complexity until a satisfactory model (as determined by our target *y-predicted* vs *y-actual* R^2 score) is found, as shown in Fig. 1. It is key to note, though, that in between these transformations, our system also reduces the complexity of our dataset by identifying mathematical symmetries. Although "AI Feynman" includes other, more minor steps in their algorithm, we chose to use this slightly simplified model in order to evaluate the core foundation laid by "AI Feynman," which we view as regression, symmetries, and data table transformation. This approach also enabled us to more clearly examine the two gaps we identified—the impacts of noise and varying data ranges—as described later.

A. Iteratively Updating the Data Table

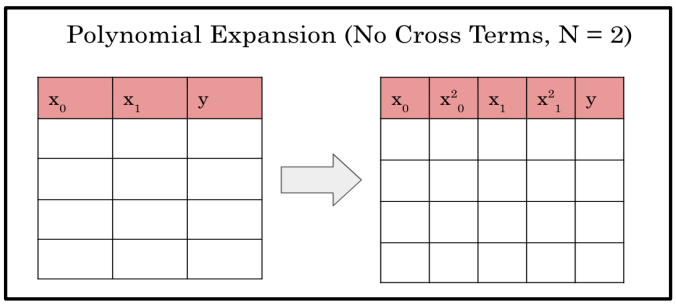


Fig. 2. Visual representation of data table expansion, showing the addition of new columns for higher powers of the independent variables.

If simple linear regression cannot fit our data, then we expand our data matrix with new features derived from the original independent variables, with the goal of transforming the problem into a solvable linear regression task. Initially, we perform univariate transformations on the dataset, creating polynomial terms up to a predefined degree, *N*, as shown in the diagram in Fig. 2. A multivariate linear regression is then performed on this expanded basis to determine if the dependent variable can be expressed as a linear combination of these new features with a sufficiently high R^2 value.

However, in some cases, the target R^2 value will not be met by this approach, as many physics equations utilize transcendental functions. While these can often be represented with Taylor series, they sometimes require a large number of terms to achieve reasonable accuracy and are difficult to interpret directly. Consequently, we have extended our approach to include new data columns (termed "functional features") whose values are determined by pre-specified functions—those commonly found in physics equations—of the original variables. For example, if we have columns x_0, x_1, and x_2, we can add columns $\sin(x_0)$, $\sin(x_1)$, and $\sin(x_2)$, after which we perform linear regression on the extended data table. If none of the above strategies work, as a last resort, our algorithm will expand the data table with cross terms—terms containing products of different independent variables up to a maximum degree, *N*.

B. Reducing Variables Through Symmetry

Considering cross terms is sometimes necessary, but it introduces a large combinatorial complexity. Therefore, we look for a way to simplify the problem by reducing the number of variables. Specifically, we take advantage of the fact that many physical laws exhibit symmetries. Broadly, a symmetry is a transformation that leaves a system unchanged. For instance, a square exhibits 90˚ rotational symmetry, since it remains the same after being rotated 90˚ about its center. For our purposes, there are certain symmetries in physics that we can exploit to reduce the number of variables. For instance, consider (1), Newton's Law of Universal Gravitational Attraction.

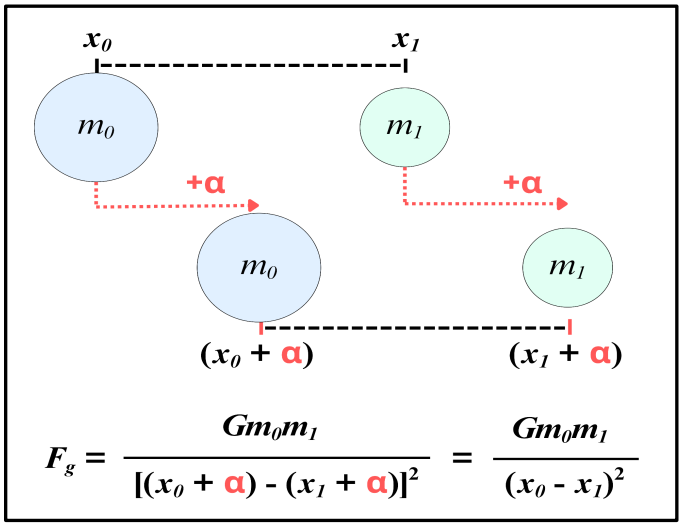


Fig. 3. Diagram showing that after a two-mass system is horizontally translated by α, the position of each mass changes, but their distance does not, leaving F_g unchanged.

Fig. 3 demonstrates that F_g is only dependent on the difference between x_0 and x_1, not their individual values. If the entire system is translated by some constant, α, the values of x_0 and x_1 change. However, since they change by the same amount, the distance between them remains the same, yielding the same F_g. This demonstrates that the system exhibits translational symmetry. More importantly, it reveals that the individual values of x_0 and x_1 are not relevant to the output; only their difference is. Therefore, in deriving a mathematical expression for F_g from data, it does not make sense to separately track the values of x_0 and x_1. We can instead simplify the problem by condensing them into a single data column, $(x_0 - x_1)$. This is the key feature of the symmetries that is useful to us: only the relationship between two variables contributes to the output.

We can generalize this to say that for any mystery function f, if $f(..., x_i, ..., x_j, ...) = f(..., x_i + \alpha, ..., x_j + \alpha, ...)$ then there exists a translational symmetry between the variables x_i and x_j. In such a case, it is not useful to individually track the values of x_i and x_j, so we condense them into a single variable that encapsulates their difference because that is all that is relevant to the output. In doing so, we reduce the number of variables, simplifying the problem.

```
function FindSymmetries(M, V)
    X ← Train MLP on M
    Initialize S ← ∅,   U ← ∅
    for each unordered pair (i, j) of input-column indices do
        if i ∈ U or j ∈ U then
            continue
        end if
        ε, a_α, m_α ← optimal_params(X_i, X_j)        ▷ Compute parameters
        if err_(i, j, a_α) < ε then
            S ∪ {(i, j, −)}, U ∪ {i, j}                ▷ Add symmetry (−)
            continue
        end if
        if err_+(i, j, a_α) < ε then
            S ∪ {(i, j, +)}, U ∪ {i, j}                ▷ Add symmetry (+)
            continue
        end if
        if err_×(i, j, m_α) < ε then
            S ∪ {(i, j, ×)}, U ∪ {i, j}                ▷ Add symmetry (×)
        end if
    end for
    Construct M′, V′ from M, V, S, U
    return (M′, V′, |S|)
end function
```

Fig. 4. Pseudocode for finding symmetries, which involves checking every pair of independent variables for translational (subtractive), sum (additive), and scale (multiplicative) symmetry, in that order, using pre-set or calculated additive and multiplicative α values; the data table is then modified accordingly. The set S keeps track of the symmetries found. The set U keeps track of the variables already used in a symmetry simplification to avoid multiple symmetries of the same variable. The *err* functions return the multilayer perceptron regressor's (MLP's) prediction error with the α-translated values. ε is the error threshold.

This simple test for translational symmetry, adding a constant to two variables, is extremely useful in reducing the complexity of the data. However, as we do not know the mystery function f, we cannot simply substitute the transformed values into a formula. Instead, we train a neural network, specifically a multilayer perceptron (MLP) regressor, on the original data such that it becomes a good approximation of f. Then, we use it to predict new outputs from the translated values. In that way, we can approximate $f(..., x_i + \alpha, ..., x_j + \alpha, ...)$ without having to know f itself. We then compare the neural network's new predictions to the original outputs, and if the error falls below a certain threshold, we conclude that there exists a symmetry between x_i and x_j, as the code in Fig. 4 shows. Note that this test is performed on all possible pairs of independent variables.

The significance of this approach becomes more apparent when considering more complex equations. Take the Law of Gravitational Attraction expanded into three dimensions.

$$F_g = \frac{Gm_0m_1}{(x_0-x_1)^2+(y_0-y_1)^2+(z_0-z_1)^2} \tag{2}$$

This equation exhibits three distinct translational symmetries: $(x_0 - x_1)$, $(y_0 - y_1)$, and $(z_0 - z_1)$. If our algorithm found all three symmetries, it could reduce the number of variables by three, making it much easier to find the equation.

Furthermore, this approach is extensible to other mathematical relationships. Subtraction is a useful case because it corresponds to a physical symmetry, translational symmetry. However, other symmetries that are useful to us are more abstract and purely mathematical. For instance, in (2), the terms m_0 and m_1 only influence F_g through their product, so we can condense them into a single variable, m_0m_1. To test for this "multiplicative" or scale symmetry, we use a neural network to check if $f(m_0, m_1, ...) = f(m_0 \cdot \alpha, m_1 \cdot 1/\alpha, ...)$. There is no common physical symmetry associated with this multiplicative relationship, but the idea of some transformation on two variables leaving the output unchanged still relates to the general definition of a symmetry. We can also extend this to addition by checking if $f(..., x_i, ..., x_j, ...) = f(..., x_i + \alpha, ..., x_j - \alpha, ...)$.

The significance of this method is that it does not approach the task of deriving equations simply with brute force. Instead, it takes advantage of our knowledge of physical laws and mathematical properties, specifically exploiting symmetries to reduce the number of independent variables. In this way, we take complex mathematical patterns and break them down into simpler relationships, making the task of deriving an equation from data feasible despite the large combinatorial space of potential expressions.

IV. Results

We successfully replicated the core of "AI Feynman" [1], and we found the system to be generally robust. (See the Appendix for a link to the data we collected.) Still, our simplified model of "AI Feynman" struggled with several categories of equations. For instance, in the formulas for average power and centripetal force, the presence of both a non-constant numerator and a non-constant denominator was difficult for our system, although our system did manage to detect the variable denominator in the oscillation-period equation. This can be explained by the fact that, although we have a "functional feature" to represent the reciprocal function $1/x$, it is applied to the data table very late in the process, by which time many other irrelevant variables could be polluting the search space for potential terms, hindering the detection of the multiplicative symmetry that would otherwise combine the numerator and denominator. While "AI Feynman" partially addresses these issues through separability and dimensional analysis strategies, our findings uniquely showcase how far the base principles that "AI Feynman" advocates for—linear regression, symmetry searching, and data table transformations—can go, and where

their limits are. That said, we followed up on these initial results by conducting further research on noise and varying data ranges, exploration that our simplified system greatly facilitated.

A. *Effect of Noise*

We examined the effects of noise by generating test data (10,000 random data points, or observations, per equation) for a variety of physics laws, primarily from classical mechanics. We then added noise sampled from a normal distribution of varying standard deviations. We tested standard deviations of 0.0 (control), 0.2, 0.4, and 0.6, which served as different "noise levels." We ran our algorithm to completion at each noise level, recording the last equation it found and the associated R^2.

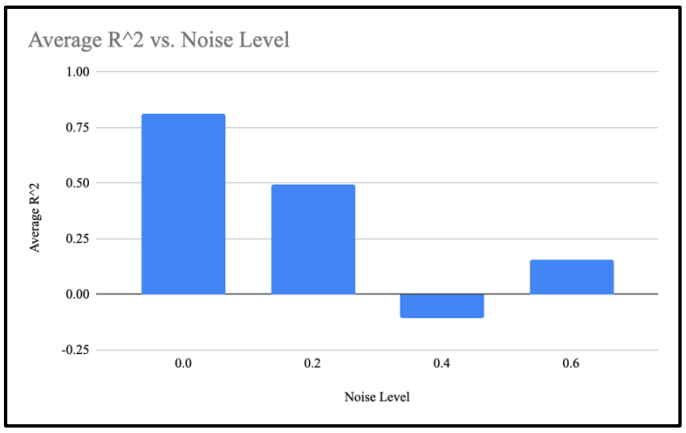


Fig. 5. Bar chart demonstrating a trend of poorer fits at higher noise levels, though, in some cases, higher noise produced overfitted equations that did not necessarily have a low R^2, despite being unhelpful (find more details with specific examples and outputs in the data linked in the Appendix).

Our equation-discovery algorithm performed noticeably worse at higher noise levels, but the effect was even more pronounced for more complex equations. For instance, for the relatively simple kinematics equation $v_f = v_i + at$, the model still found $v_f = 0.9951v_i + 1.0209at + 0.0897$, an approximately correct solution with $R^2 = 0.99$, at a noise level of 0.4. However, for the oscillation-period equation, $T = 2\pi/\omega$, which contains a more complex inverse function, our equation-finding system collapsed at a noise level of just 0.2, outputting a garbled equation with $R^2 = 0.003$. The equation-finding algorithm similarly struggles with equations with several multiplied terms of higher powers at higher noise levels. For example, in the classical kinetic energy equation $K = \frac{1}{2}mv^2$, the fit drops from $R^2 = 0.996$ at a noise level of 0.2 to $R^2 = 0.045$ at a noise level of 0.4. In summary, the equation-fitting system collapses more quickly (i.e., at lower noise levels) for more complex functions, though the delineation of what is "complex" is somewhat arbitrary. (The Appendix lists these and other examples in more detail, but Fig. 5 provides an overview of the general trend.)

Regarding the specific effects of noise, our system occasionally suffers from underfitting and overfitting when dealing with different noise levels. For a case like the mechanical-energy equation $E = U_g + U_s + K$, at a noise level of 0.6, our algorithm underfitted and just outputted a constant, disregarding the independent variables: $E = 0.6082$. Conversely, in trying to fit the equation for the gravitational potential energy of an object near the surface of a massive body, $U_g = mgh$, our algorithm severely overfit at high noise levels, outputting an equation with 18 terms and trigonometric functions, which can be found in the data linked in the Appendix.

B. *Adapting Alpha (α) to Varying Numerical Ranges*

Another practical consideration is that real-world datasets have varying numerical scales, impacting the ideal α values for those datasets. For instance, on smaller-ranged datasets (e.g., with independent variables ranging from -1 to 1), $\alpha = 5$ may cause the symmetry subroutine to reject true symmetries because the MLP regressor could not extrapolate so far beyond its training. Meanwhile, for larger-ranged datasets (e.g., with independent variables ranging from -100 to 100), $\alpha = 5$ may not significantly change the MLP regressor's predicted output regardless of whether there is a symmetry or not, resulting in false-positive symmetry detections.

We thus explored scaling α proportionally to the standard deviation of the columns being tested for additive symmetries, and in proportion to the standard deviation of the order of magnitude of the columns being tested for multiplicative symmetries (as multiplication impacts numbers on the order-of-magnitude scale). The code for the adaptive α is linked in the Appendix. We tested equations containing both true and fake symmetries using both the fixed-constant approach (setting *additive* $\alpha = 5$ and *multiplicative* $\alpha = 2$) and our novel adaptive approach on uniformly distributed data with ranges from -0.5 to 0.5, -1 to 1, -10 to 10, -50 to 50, and -100 to 100. We recorded all symmetries detected, true or false, within the first cycle of iterative symmetry-based variable reduction for each equation.

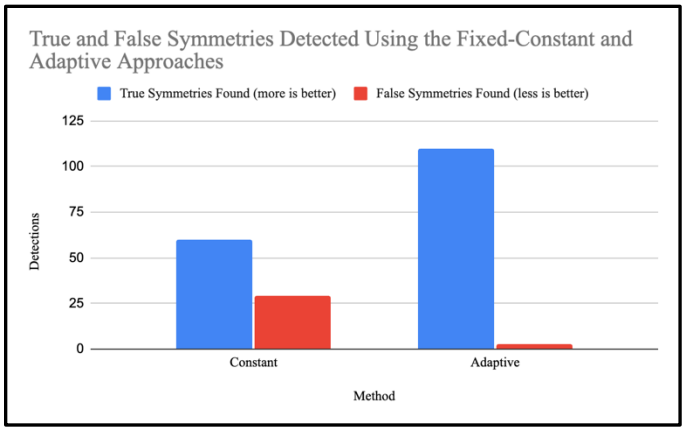


Fig. 6. Bar chart demonstrating that, on our testing datasets, our novel adaptive method detected significantly more true symmetries across the data ranges while reporting only three false symmetries in total, significantly outperforming the traditional fixed-constant approach on both metrics.

Empirically, the adaptive approach was much better in automatically dealing with both larger-ranged and smaller-ranged datasets (see Fig. 6 for an aggregated summary, or see the raw data linked in the Appendix). Moreover, with the fixed constant approach of *additive* $\alpha = 5$, we found that symmetry searching was most successful on uniformly distributed data ranging from -10 to 10, for which our additive adaptive-alpha formula predicts a very similar ideal *additive* α of about 5.8. These findings provide confidence that our adaptive α approach is promising and impactful.

V. Conclusion

This work replicates a few key components of the "AI Feynman" algorithm [1] using a "build-from-scratch" approach. We implemented linear regression, polynomial transformations, and symmetry analysis. As a proof-of-concept, as well as to handle a larger variety of equations, we also extended that core functionality to handle cross terms and non-polynomial elementary functions. Finally, we most directly expanded on the work of "AI Feynman" by investigating and classifying noise more thoroughly and by finding a formula to adapt symmetry α transformations to various data scales. Our noise experiments found that the system performs with lower accuracy at higher noise levels, especially with more complicated functions. Specifically, many multiplied variables, variables in the denominator, variables with higher powers, and other complex characteristics seem to lead to greater sensitivity to noise, as even slight noise could cause significant deviations from such an intricate underlying relationship. The main effects of noise can be categorized as "underfitting" or "overfitting," which manifest in forms described earlier in the paper. This deeper understanding of the specific effects of noise, as well as what types of equations noise most impacts, is of utmost importance because real-world data, collected from imperfect instruments, is almost always noisy. Furthermore, we found that symmetry detection can significantly simplify equation fitting by reducing the number of independent variables to consider. Moreover, symmetries can provide deeper insight into the structure of an equation and, ultimately, our physical reality itself. Thus, any improvements to symmetry detection, like our adaptive α formulas, have far-reaching benefits.

Regarding limitations, our current system's symmetry detection may (albeit very rarely, per our data) find false positives or non-existent symmetries, perhaps due to an overly permissive (too small α) symmetry test or, more likely, a poor MLP model. Though our adaptive α formulas significantly reduce the false-positive rate, as Fig. 6 illustrates, they do not yet completely eliminate false positives. Our system also struggles with functions containing non-integer powers (e.g., cube roots) or nested compositions (e.g., $\sin(\exp((1/x)))$ because they are not represented well in our extended data tables. The "functional features" technique offers a partial solution, enabling us to consider arbitrary combinations of functions, including fractional exponents. However, to use functional features, we have to explicitly define the functions the algorithm should consider (since there are infinitely many possible mathematical functions), which is not ideal.

A potential extension of our work could be to incorporate log analysis, where we would take the log of the variables before running the main algorithm. This should enable the discovery of arbitrary exponents, which would be the coefficients of the log-modified terms. Additionally, future studies could involve tuning the threshold of success (i.e., the cut-off score after which the system stops trying more complex fits) based on the suspected noise level. Such an early-stopping strategy may partially combat the effects of noise by accepting simpler functions that technically have a "worse fit," considering that many physics equations are typically low-order polynomials or trivial combinations of elementary functions. Future work should also seek to improve AI-powered symmetry detection, perhaps expanding on our adaptive formulas or investigating the merits of other neural networks. Ultimately, we believe that though much prior work has been done on the theoretical aspects of equation fitting, shifting attention to practical considerations is the next step to integrate sophisticated symbolic equation fitting into mainstream science.

Appendix

All our code was written in a Google Colab file, which has been exported as a Jupyter Notebook. All our results and analyses were recorded in a Google Sheets file, which has been exported as a PDF. All of these materials can be accessed through the following GitHub repository, which describes how to locally download or view online each of the materials: https://github.com/AI-PHYSICS-IMPROVEMENTS-CONSIDERATIONS/AI-ASSISTED-DISCOVERY-OF-PHYSICS-LAWS-PRACTICAL-IMPROVEMENTS-AND-CONSIDERATIONS

Acknowledgment

We thank David Hoyt, Dr. Kouh, and the Governor's School of New Jersey Program in the Sciences (GSNJS) at Drew University.

References

[1] S.-M. Udrescu and M. Tegmark, "AI Feynman: A physics-inspired method for symbolic regression", *Science Advances*, vol. 6, no. 16, p. eaay2631, Apr. 2020, doi: 10.1126/sciadv.aay2631.

[2] P. W. Langley, 'BACON: a production system that discovers empirical laws', in *Proceedings of the 5th International Joint Conference on Artificial Intelligence - Volume 1*, Cambridge, USA, 1977, p. 344.

[3] B. G. Cohen, B. Beykal, and G. M. Bollas, "Physics-informed genetic programming for discovery of partial differential equations from scarce and noisy data," *Journal of Computational Physics*, vol. 514, p. 113261, Oct. 2024, doi: 10.1016/j.jcp.2024.113261.

[4] Kamienny, P., et al, "Deep generative symbolic regression with Monte-Carlo-tree-search," in *Proceedings of the 40th International Conference on Machine Learning*, 2023.

[5] R. P. Feynman, R. B. Leighton, and M. Sands, *The Feynman Lectures on Physics*, vol. 1. Reading, MA, USA: Addison-Wesley, 1964.